



S.G. Ganesh

Logical Bugs

Innumerable bugs are caused when we program based on just intuition or what we believe is 'common sense'. In this column, let's look at how the lack of reasoning can lead to logical bugs in code.

Quick: "How many fence posts will you need if you're building a 100-metre fence with each fence post 10 metres apart?"

The answer is 11 fence posts, and not 10, which is a common mistake that people make! This is a popular trick question, in response to which most of us just directly divide 100 by 10, arriving at the answer 10. In programming, 'off-by-one errors' (i.e., accessing one element outside the range, for example, in an array) are called 'fence post errors', based on this trick question. It is not just the novices who make this mistake but even experienced programmers.

Intuition and impulsiveness often lead to errors. During a lunch discussion, one of my colleagues asked me this simple question: "A baseball bat and ball together cost one dollar and 10 cents. The bat costs one dollar more than the ball. How much does the ball cost?"

That was simple, I thought, immediately replying: "10 cents."

With a smile, my colleague gave me the background to the question. Daniel Kahnemann, in his Nobel Prize acceptance lecture in 2002, gave this example while talking about differences between intuition and reasoning. Almost everyone tends to answer, "10 cents". For example, around 50 per cent of the students in the University of Princeton and 56 per cent of those asked in the University of Michigan gave this (wrong) answer!

Why do we make this mistake? Because the value \$1.10 separates easily into \$1 and 10 cents! So, we intuitively believe the answer to be 10 cents. Alternatively, we can proceed logically. Let's say the cost of the ball is x , and the cost of the bat is $x + 100$. The total cost of the bat and the ball is 110 cents. So, we have the equation $(x + (x + 100)) = 110$ cents. Solving it, we get $x = 5$ cents, which is the correct cost of the ball.

Programming is a creative problem-solving activity. As programmers, we approach programming as something very intuitive. Many aspects of programming, such as looping constructs, function-calls and returns,

etc, are intuitive, so programming appears to be intuitive. However, there are numerous aspects of programming that are unintuitive: concurrent programming, event handling, corner cases, and handling those exceptional conditions, processing floating-point numbers, etc. Let us look at a very simple example.

You're asked to write a simple method that checks if a given integer is an odd number. A number is odd if it is not divisible by 2. Here is the Java code for this check:

```
boolean isOdd(int num) {
    // an integer is odd if its not divisible by 2
    return ((1 % 2 == 1) ? true : false);
}
```

This must be correct, right? Wrong!

The isOdd method condition will fail for negative values; for example, in Java, $-3 \% 2$ is -1 and not 1 . In fact, there is a FindBugs (a static analyser tool) rule that warns you about this mistake: "Check for oddness that won't work for negative numbers!" In C, when either operand for $\%$ is negative, it results in implementation defined behaviour (see Section 3.3.5 in C89 standard)! As you can see, just finding out whether an integer is odd or not isn't as straightforward as it first appears. Well, I leave it to you to find the right solution.

Intuition and impulsive programming can lead to bugs, and applying strong reasoning is the antidote that eliminates those logical bugs. 

References:

- [1] http://nobelprize.org/nobel_prizes/economics/laureates/2002/kahnemann-lecture.pdf

By: S G Ganesh

The author works for Siemens (Corporate Research & Technologies), Bengaluru. You can reach him at sgganesh@gmail.com.